2026

Scope of Work Document

POSTBUILDER

ALEJANDRA BADIA

DOCUMENT REVISION LOG

REV.	UPDATE DATE	BY
1	MARCH 10, 2026	ALEJANDRA

Contents

1. Project Overview 3
2. Project Objectives 3
3. Technology Stack 3
3.1 Frontend 3
5.2 Architecture Approach 4
5.3 Backend 4
4. Application Architecture 4
5. Application Layout 5
5.1 Primary Layout 5
5.2 Modal System 5
6. Core Application Features 5
7. Form Sections 6
7.1 Post Details 6
7.2 Author 6
7.3 SEO Settings 6
7.4 Tags 6
7.5 Media Upload 7
7.6 Publish Settings 7
8. Live Blog Preview 7
9. Validation System 7
10. JSON Output Generator 8
11. Submission Workflow 9
12. Modal System 9
12.1 Upload Modal 9
12.2 Intro Modal 10
13. Developer Utility Features 10
13.1 Copy JSON 10
13.2 Download JSON 10

14. UI Design Features 10
14.1 Design System 10
14.2 Layout 10
15. Folder Structure 11
16. Documentation 11
17. Future Improvements 11

1. Project Overview

PostBuilder will be a lightweight web application prototype designed to simulate a modern content publishing workflow similar to those used in Content Management System (CMS) platforms.

The application will allow users to create blog post data through a structured multi-step form interface and will generate a JSON payload representing the post. It will include a live preview, validation system, and a simulated publishing workflow using asynchronous requests.

The project will focus on demonstrating:

- Structured content modeling
- Frontend workflow design
- Real-time UI feedback
- JSON payload generation for API workflows
- CMS-style publishing experience
- Asynchronous form submission with progress feedback

2. Project Objectives

The goal of PostBuilder will be to demonstrate understanding of modern product platform patterns including:

- Structured content creation workflows
- Real-time UI feedback for editors
- Frontend validation using native browser APIs
- Multi-section form navigation and state handling
- JSON data modeling for API integrations
- Asynchronous submission and UI feedback patterns

The application should resemble a simplified CMS editor experience.

3. Technology Stack

3.1 Frontend

- HTML
- CSS
- JavaScript (vanilla)

5.2 Architecture Approach

- Modular JavaScript files
- Component-style layout
- CSS variables for design system

5.3 Backend

- PHP endpoint for JSON generation
- No database processing

4. Application Architecture

The application will be built as a single-page interface with structured UI components.

Workflow architecture:

User Input
↓
Form Sections (Multi-Step UI)
↓
Native Validation Layer (HTML5 + custom handling)
↓
AJAX Submission (XMLHttpRequest)
↓
Server JSON Generator (PHP)
↓
Response Handling (UI updates + Modal + JSON Panel)
↓
Live Preview Renderer

This architecture will demonstrate frontend-driven workflows with lightweight backend processing.

5. Application Layout

The primary layout structure includes a sidebar navigation, with a main area that holds a split view of the form and the preview sections, and a JSON panel that appears if a form is successfully processed.

5.1 Primary Layout

Sidebar Navigation | Form Section | Live Preview

JSON Output Panel

5.2 Modal System

- Welcome modal (Introduction)
- Upload status modal with either upload complete status or errors, if any

6. Core Application Features

The application will use sidebar navigation to move between sections.

Sidebar sections will include:

- Post Details
- Author
- SEO
- Tags
- Media
- Publish Settings

Each section will load a corresponding form panel.

Active state will be synchronized between:

- Form sections
- Sidebar navigation
- Validation feedback

7. Form Sections

7.1 Post Details

Fields will include:

- Post Title (required)
- Slug (auto-generated, read-only)
- Short Description (required)
- Post Content (required)

Slug generation rules will include:

- Lowercase formatting
- Spaces replaced with hyphens
- Removal of special characters

7.2 Author

Fields will include:

- Author Name (required)
- Author Role (optional)
- Author Bio (required)

7.3 SEO Settings

Fields will include:

- Meta Title (optional)
- Meta Description (optional)

7.4 Tags

The application will include a custom tag input system:

- User-defined tags (optional)

Tag input will include a dynamic tag UI and storage via form state

7.5 Media Upload

The application will support a featured image upload.

Validation will include:

- File type restrictions (JPG / PNG)
- Client-side preview
- Size handling via backend constraints and UI

7.6 Publish Settings

Fields will include:

- Publish preference (Anytime / ASAP / Specific Date)
- Conditional date input (dynamically displayed)

8. Live Blog Preview

A real-time preview will update as the user enters content.

The preview will display:

- Title
- Description
- Author
- Content
- Image
- Tags
- Publish date
- About the author section

This will demonstrate:

- DOM manipulation
- Real-time UI synchronization
- Product-oriented preview design

9. Validation System

The application will use native HTML5 validation enhanced with custom logic.

Validation features will include:

- Required field enforcement
- Character limits (maxlength)
- Pattern validation
- Custom invalid event handling

Custom behavior will:

- Automatically navigate to the section containing invalid fields
- Synchronize sidebar active state
- Scroll to the relevant section
- Focus the invalid input
- Display a global error message

---

10. JSON Output Generator

The system will generate a structured JSON payload based on user input.

Example structure:

```
{
  "title": "...",
  "slug": "...",
  "description": "...",
  "content": "...",
  "tags": [],
  "author": {
    "name": "...",
    "role": "...",
    "bio": "..."
  },
  "metaTitle": "...",
  "metaDescription": "...",
```

```
  "featuredImage": "...",
  "publishPreference": "...",
  "specificPublishDate": "..."
}
```

The JSON will be displayed in a formatted preview panel after submission.

11. Submission Workflow

When the user clicks Publish Post, the system will:

- Trigger native validation
- Handle invalid fields with guided navigation
- Submit the form asynchronously via AJAX if valid
- Perform backend validation
- Display an upload modal with progress feedback
- Process data on the server and return a JSON response, if no error(s) exist
- If error(s) exist, error(s) will be displayed via modal after form submission and after all validations are confirmed via backend

12. Modal System

12.1 Upload Modal

Will include:

- Progress bar
- Percentage indicator
- Success/error messaging

12.2 Intro Modal

Will provide:

- Onboarding guidance for first-time users

13. Developer Utility Features

13.1 Copy JSON

A button will allow users to copy the generated JSON payload to the clipboard.

13.2 Download JSON

A button will allow users to download the JSON file (post.json).

14. UI Design Features

14.1 Design System

CSS variables will be used for:

- Colors
- Spacing
- Typography
- Border radius

14.2 Layout

The UI will include:

- Sidebar navigation with active state highlighting
- Card-based form sections
- Live preview panel
- Modal overlays

15. Folder Structure

```
postbuilder/
|— index.php
|— api/
|   └— jsonGenerator.php
|— assets/
|   ├— css/
|   ├— js/
|   └— images/
|— layouts/
|   ├— header.php
|   └— footer.php
```

16. Documentation

The repository will include:

- Project overview
- Features
- Architecture
- Setup instructions
- Future improvements

17. Future Improvements

Potential enhancements may include:

- Database persistence
- CMS API integration
- Markdown editor support
- Autosave improvement
- Multi-user workflows and role-based permissions